

CMSC 433: Midterm Exam 2 (Spring 2025)

Question 1 (20 points)

For each of the Haskell expressions below, write their (most general) Haskell type or “ill-typed” if it contains a type error. For the following problems, the definition of `State` is given below. The type signatures of all other functions below are provided in the appendix at the end.

```
newtype State s a = S {runState :: s -> (a, s)}
get  :: State s s
put  :: s -> State s ()
modify :: (s -> s) -> State s ()
```

(a) `get >>= put`

Example answer:

`State s ()`

(b) `\l -> [ x | x <- l, x ]`

(c) `\l -> [ even x | x <- l ]`

(d) `modify tail`

(e) `put 42 >> put True`

(f) `(,) <$> get`

Question 2 (20 points)

For each of the types below, write the most well-defined Haskell expression that has that type. That is, don't write trivial expressions (such as `[]`, `Nothing`, or `undefined`) unless there is no other option. You can use any function from the appendix, functor/applicative operations, or list comprehensions.

(a) `Int -> Int`

Example answers:

`\x -> x + 1`

`(+1)`

(b) `[Maybe a] -> Maybe [a]`

(c) `Monad m => [a] -> (a -> m b) -> m [b]`

(d) `(a -> b -> c) -> b -> State a c`

(e) `Monad m => m a`

(f) `State (a,b) (b,a)`

Question 3 (20 points)

Recall from the previous midterm the notion of triangle numbers: A *triangle* number counts objects arranged in an equilateral triangle. The n th triangular number is equal to the sum of the n natural numbers from 1 to n . We can implement triangle numbers as a Dafny function:

```
function Triangle(n:int) : int {
  if n <= 0 then 0 else n + Triangle(n-1)
}
```

Consider the following Dafny program that calculates this sum, incrementing the final result by one:

```
method Tri(n:int) returns (t:int)
  requires n > 0
  ensures t == Triangle(n) + 1;
{
  t := 0;
  var i : int := n;
  while i > 0
  {
    t := t + i;
    i := i - 1;
  }
  t := t + 1;
}
```

For each of the following three invariants that were proposed during the first midterm, list the induced verification conditions.

(a) $i \geq 0 \ \&\& \ t + \text{triangle}(i) == \text{triangle}(n)$

(b) $i \geq 0 \ \&\& \ t == \text{triangle}(n-i)$

(c) $t == \text{triangle}(i)$

Question 4 (20 points)

(a) Consider the following Haskell program:

```
foo :: Monad m => [m (a->b)] -> m [(a->b)]
foo [] = return []
foo (m:ms) = do
  f <- m
  fs <- foo ms
  return (f:fs)
```

Evaluate the following expressions involving `foo` (you can leave functions that don't evaluate further such as `id` or `const 42` as is):

`foo [[]] =`

`foo [Just id] =`

`foo [Just id, Nothing] =`

`foo [[id, const 42],[const 3]] =`

`runState (foo [get, get]) id =`

(b) Recall the `mapM` function from the standard library, with type `mapM :: Monad m => (a -> m b) -> [a] -> m [b]`. This is the monadic counterpart of `map` which iterates through a list and maps the monadic action over it, gathering its results.

Implement `foo` in terms of `mapM`.

```
foo :: Monad m => [m (a -> b)] -> m [(a -> b)]
foo
```

Question 5 (20 points)

(a) Consider the following Haskell program:

```
foo :: [Int] -> Int -> (Int, [Int])
foo [] m = (m, [])
foo [x] m = (x, [m])
foo (l : ls) m = (min m' l, m : ls')
  where (m', ls') = foo ls m
```

Evaluate the following expressions involving `foo`:

`foo [] 42 =`

`foo [1] 17 =`

`foo [1,2,3,4] 42 =`

`foo [1..10] 0 =`

(b) Using `foo`, implement a `toMin` function, which given a list of integers return the list with all of its elements replaced by its minimum element. For example:

```
toMin :: [Int] -> [Int]
toMin [] = []
toMin [1,2,3,4] = [1,1,1,1]
toMin [17,42,0] = [0,0,0]
```

Implement `toMin` in terms of `foo`. Do not use explicit recursion; do not use other recursive functions (no folds or other recursive functions such as `minimum`).

You can achieve this in a straightforward manner with 2 calls to `foo` and `let` binding or `fst/snd`.

BONUS: Can you do it with just 1 call to `foo`?

```
toMin :: [Int] -> [Int]
toMin l =
```